

1.Implement the following data structures.**A) Linked List.****CODE:**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def append(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def prepend(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
    def delete(self, data):
        current_node = self.head
        if current_node is not None and current_node.data == data:
            self.head = current_node.next
            current_node = None
            return
        previous_node = None
        while current_node and current_node.data != data:
            previous_node = current_node
            current_node = current_node.next
        if current_node is None:
            return
        previous_node.next = current_node.next
        current_node = None
    def display(self):
        current_node = self.head
        while current_node:
            print(current_node.data, end="->")
            current_node = current_node.next
        print("None")
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.append(1)
    linked_list.append(2)
    linked_list.append(3)
```

```
linked_list.append(0)
linked_list.display()
linked_list.delete(1)
linked_list.display()
```

OUTPUT:

```
>> 1->2->3->0->None
    2->3->0->None
```

B) Stack.**CODE:**

```
class Stack:
    def __init__(self):
        self.items=[]
    def is_empty(self):
        return len(self.items)==0
    def push(self,item):
        self.items.append(item)
    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("Pop from an empty stack.")
    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("Peek from an empty stack.")
    def size(self):
        return len(self.items)

if __name__=="__main__":
    stack=Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print("Stack size:",stack.size())
    print("Top of the Stack:",stack.peek())
    print("Popped Element:",stack.pop())
    print("Stack size after the pop:",stack.size())
```

OUTPUT:

```
Stack size: 3
Top of the Stack: 3
Popped Element: 3
Stack size after the pop: 2
>>> |
```

C) Queue.**CODE:**

```
class Queue:
    def __init__(self):
        self.items=[]
    def is_empty(self):
        return len(self.items)==0
    def enqueue(self,item):
        self.items.append(item)
    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue.")
    def peek(self):
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("Pack from an empty queue.")
    def size(self):
        return len(self.items)
if __name__=="__main__":
    queue=Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print("Queue size:",queue.size())
    print("Front element:",queue.peek())
    print("Dequeue element:",queue.dequeue())
    print("Queue size after dequeue:",queue.size())
```

OUTPUT:

```
Queue size: 3
Front element: 1
Dequeue element: 1
Queue size after dequeue: 2
>>> |
```

Assignment-1: Reverse a linked list.**CODE:**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def reverse(self):
        previous_node=None
        current_node=self.head
        while current_node:
            next_node=current_node.next
            current_node.next=previous_node
            previous_node=current_node
            current_node=next_node
        self.head=previous_node
    def display(self):
        current_node = self.head
        while current_node:
            print(current_node.data, end=" ")
            current_node = current_node.next
        print()
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.append(1)
    linked_list.append(2)
```

```
linked_list.append(3)
linked_list.append(4)
print("Linkedlist:")
linked_list.display()
linked_list.reverse()
print("Reverse linked list is:")
linked_list.display()
```

OUTPUT:

```
Linkedlist:
1 2 3 4
Reverse linked list is:
4 3 2 1
>>> |
```

Assignment-2: Reverse a stack.**CODE:**

```
class Stack:
    def __init__(self):
        self.items=[]
    def is_empty(self):
        return len(self.items)==0
    def push(self,item):
        self.items.append(item)
    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("Pop from an empty stack.")
    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("Peek from an empty stack.")
    def size(self):
        return len(self.items)
    def display(self):
        return self.items
    def reverse(self):
        return self.items[::-1]

if __name__=="__main__":
    stack=Stack()
    stack.push(1)
```

```
stack.push(2)
stack.push(3)
print("Original stack:",stack.display())
print("Reverse of stack:",stack.reverse())
```

OUTPUT:

```
>> | Original stack: [1, 2, 3]
    | Reverse of stack: [3, 2, 1]
    |
```